**FH-OÖ Hagenberg/HSD**
**SDP3, WS 2026**
***Übung 4***

Name: T. Lechner, P. Ackermann | Aufwand in h: 12

Mat.Nr: S2510306017, S2510306001 | Punkte:

Übungsgruppe: 1 | korrigiert:

**Beispiel 1 (24 Punkte) Bilderverwaltung:** Erstellen Sie eine Software zur Erzeugung und Verwaltung von Bildern mit graphischen Objekten. Die Bilder sind in einer Bilderverwaltung abgelegt und können durch generieren einer SVG-Datei in einem entsprechenden Browser angezeigt werden.

Eine Methode `WriteSVG` soll das Bild als SVG-Datei (*scalable vector graphics*) mit dem gegebenen Namen abspeichern. Der Name der SVG-Datei wird bei der Konstruktion eines Bildes festgelegt. Ein Beispiel für eine solche SVG-Datei könnte z.B. folgendermaßen aussehen:

```
<?xml version="1.0"?>
<svg xmlns="http://www.w3.org/2000/svg">
<rect x="10" y="10" width="90" height="70" stroke="black" fill="none"/>
<circle cx="55" cy="45" r="35" stroke="black" fill="none"/>
</svg>
```

Die SVG-Datei beschreibt die Bestandteile der Grafik (im obigen Beispiel ein Rechteck und ein Kreis). Stroke (`stroke="black"`) gibt die Farbe an und `fill` die Füllung. Wird `fill="none"` weggelassen, so wird das Objekt mit der angegebenen Farbe gefüllt.

Es sollen gefüllte und nicht gefüllte Kreise und Rechtecke erzeugt werden, wobei ein Bild nur gefüllte oder nur nicht gefüllte Objekte haben kann.

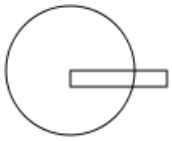


*Bild1:test1.svg*



*Bild2: test2.svg*

Die Bilderverwaltung und die Fabriken zur Objekterzeugung exisitieren nur einmal und werden beim

Beenden des Programmes automatisch freigegeben. Verwenden Sie zur Implementierung eine vorgestellte Methode aus der Vorlesung. Graphische Objekte werden mit einer entsprechenden Fabrik erzeugt und das Kopieren und Zuweisen soll verhindert werden!

Die Bilderverwaltung kann mit verschiedenen Fabriken arbeiten und so verschiedene Bilder erzeugen. Die Fabriken in der Bilderverwaltung sollen austauschbar sein!

Das Design soll so ausgelegt sein, dass in Zukunft nicht nur gefüllte und nicht gefüllte graphische Objekte erzeugt werden können, sondern auch z.B. schraffierte oder gepunktete graphische Objekte.

Für die SVG-Datei muss keine Grammatik angegeben werden!

Entwerfen Sie ein Klassendiagramm und verwenden Sie dazu entsprechende Design Pattern. Implementieren Sie das vollständige Klassendiagramm und schreiben sie einen Testtreiber. Verfassen Sie weiters eine Systemdokumentation (Funktionalität, Klassendiagramm, Schnittstellen der beteiligten Klassen, etc.)! Geben Sie die vom Testtreiber erzeugten SVG-Dateien sowohl textuell als auch graphisch ab.

***Allgemeine Hinweise:*** Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine **saubere Strukturierung** und auf eine **sorgfältige Ausarbeitung!** Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab!




# Systemdokumentation Projekt Bilderverwaltung

**Version 1.0**

T. Lechner, P. Ackermann

Hagenberg, 5. November 2026

# Inhaltsverzeichnis

**1 Organisatorisches** **5**
1.1 Team . . . . . 5
1.2 Aufteilung der Verantwortlichkeitsbereiche . . . . . 5
1.3 Aufwand . . . . . 6

**2 Anforderungsdefinition (Systemspezifikation)** **6**

**3 Systementwurf** **7**
3.1 Klassendiagramm . . . . . 7
3.2 Designentscheidungen . . . . . 7

**4 Dokumentation der Komponenten (Klassen)** **9**

**5 Testprotokollierung** **9**

**6 Quellcode** **9**

# 1 Organisatorisches

## 1.1 Team


- T. Lechner, Matr.-Nr.: S2510306017, E-Mail: s2510306017@students.fh-hagenberg.at
- P. Ackermann, Matr.-Nr.: S2510306001, E-Mail: s2510306001@students.fh-hagenberg.at


## 1.2 Aufteilung der Verantwortlichkeitsbereiche

- T. Lechner
  - Klassendiagramm
  - Designentscheidungen
  - Implementierung und Unit-Test:
    - ImageManager
    - Image
    - GraphicObject
    - Rectangle
    - FramedRectangle
    - FilledRectangle
- P. Ackermann
  - Klassendiagramm
  - Anforderungsdefinition
  - Implementierung und Unit-Test:

* Object
* SingletonBase
* GraphicObjectFactory
* FramedObjectFactory
* FilledObjectFactory
* Circle
* FramedCircle
* FilledCircle

## 1.3 Aufwand

- T. Lechner: geschätzt 7 Ph / tatsächlich 0 Ph
- P. Ackermann: geschätzt 7 Ph / tatsächlich 0 Ph

# 2 Anforderungsdefinition (Systemspezifikation)

Es soll eine Software zur Verwaltung von *Graphic Objects* in Bildern erstellt werden. Diese *Bilder* sollen im Format SVG mit einem zugehörigen Namen gespeichert werden. Ein *Bild* kann mehrfach die *Graphic Objects* Rechteck und Kreis enthalten, wobei diese Objekte durch verschiedene Eigenschaften wie die Position oder den spezifischen Größen der Dimensionierung beschrieben werden. Des Weiteren soll jedes *Graphic Object* in einem gewissen 'Stil' dargestellt werden können. Es sollen entweder gefüllte oder nicht gefüllte Rechtecke oder Kreise erstellt werden können. Allerdings sollte ein *Bild* nur *Graphic Objects* von **einem** 'Stil' enthalten können. Somit soll die Software auch für spätere Erweiterung von Stilen implementiert werden.

# 3 Systementwurf

## 3.1 Klassendiagramm

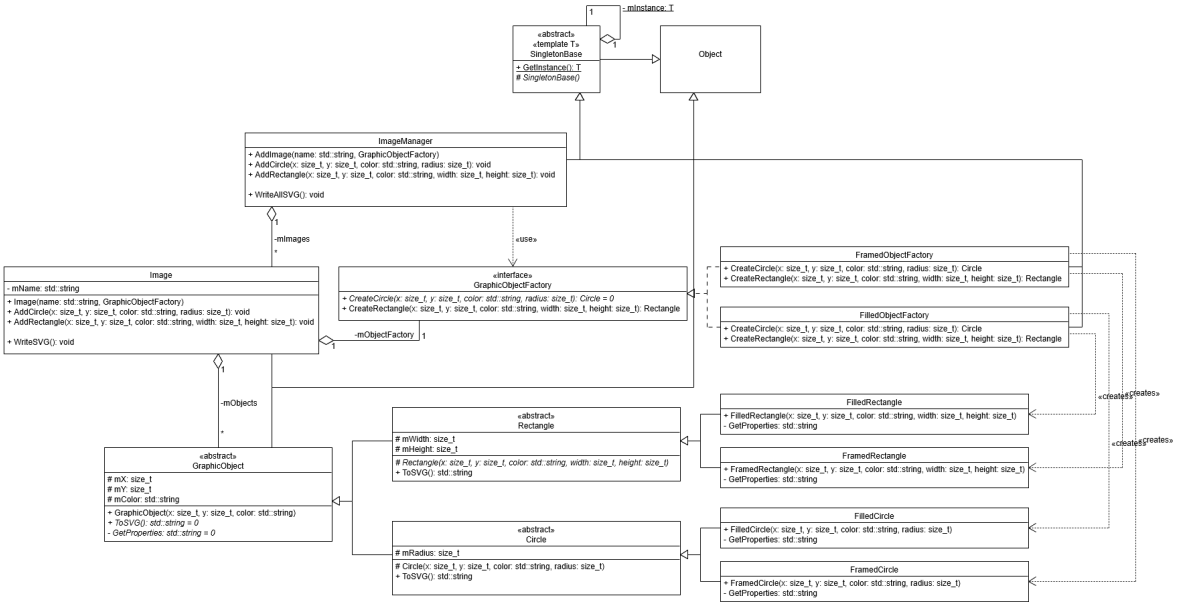



Abbildung 1: Klassendiagramm Bilderverwaltung

## 3.2 Designentscheidungen

**Object** wird wieder als Basisklasse aller Basisklassen verwendet, damit der virtuelle Destruktor überall vorhanden ist.

Die **ImageManager** Klasse ist die Hauptklasse der Anwendung, die dann vom Client verwendet wird. Man kann ein neues **Image** mit einer gewissen **GraphicObjectFactory** anlegen und dann immer auf dem neu angelegten **Image** neue **Circle**s und **Rectangle**s hinzufügen, bis ein neues **Image** hinzugefügt wird.

**Circle** und **Rectangle** erben von der abstrakten **GraphicObject** Klasse und sind aber auch selbst abstrakt, da sie noch jeweils von **FilledCircle**/**FramedCircle** bzw. **FilledRectangle**/**FramedRectangle** abgeleitet werden.

**GraphicObject** hat als Member nur die Position (X,Y) und die Farbe, **Circle** zusätzlich noch einen Radius und **Rectangle** zusätzlich Höhe und Breite.

Die Grafikobjekte werden immer von einer **GraphicObjectFactory** (abgeleitete Klassen **FilledObjectFactory** und **FramedObjectFactory**) konstruiert. **FilledObjectFactory** konstruiert immer die **Filled**-Varianten und **FramedObjectFactory** die **Framed**-Varianten.

Beim Konstruieren eines **Images** wird eine solche Fabrik mitgegeben. Mit dieser werden in *AddCircle* und *AddRectangle* neue Grafikobjekte erstellt, die intern in einer Liste gespeichert werden → **Abstract Factory Pattern**.

Die Fabriken und **ImageManager** sollen nur ein Mal gleichzeitig existieren können, dafür wird das **Singleton Pattern** verwendet. Genauer gesagt wird von der abstrakten **SingletonBase** Template-Klasse geerbt. Diese erbt von **Object**, da sie auch einen virtuellen Destruktor benötigt.

Zur Umwandlung der Grafikobjekt-Daten ins SVG-Format, wird die *ToSVG* Methode eines **GraphicObject**s verwendet, die pure virtual ist und von **Circle** und **Rectangle** implementiert wird. Dabei werden die Daten als XML-SVG-String zurückgegeben. Da die Kindklassen von **Rectangle** und **Circle** eventuell zusätzliche Eigenschaften zum XML hinzufügen müssen (z.B. fill="none"), gibt es noch eine weitere pure virtual Methode *GetProperties*. Diese wird in den untersten Klassen implementiert und von den abstrakten **Circle** und **Rectangle** in deren *ToSVG* Methode aufgerufen. Dadurch kann die Erstellung des XMLs zusätzlich angepasst werden → **Template Method Pattern**.

In *WriteSVG* generiert **Image** so die XML-SVG-Strings für jedes gespeicherte **GraphicObject** und schreibt diese Strings zusätzlich mit einem XML-Header und XML-Footer in eine Datei. Der Name der Datei entspricht dem Namen des **Image**s (Member *mName*) mit .svg Endung. **ImageManager** ruft in *WriteAllSVG* für jedes gespeicherte **Image** die *WriteSVG* Methode auf.

# 4 Dokumentation der Komponenten (Klassen)

Die HTML-Startdatei befindet sich im Verzeichnis ./../doxy/html/index.html

# 5 Testprotokollierung

# 6 Quellcode